

THE 20:26 PROJECT

GraphShield: Explainable Temporal Graph Neural Networks for Cryptocurrency Fraud Detection

Faisal Bin Basha (904178711), Mewael Hailu (904205074)

1. Introduction, Background, Motivation

1.1 What we tried to do (no jargon)

Bitcoin transactions are public — anyone can see who sent money to whom and when. Criminals exploit this openness anyway, hiding illegal payments inside the noise of millions of normal transactions. Banks and law enforcement want to flag suspicious transactions automatically, but two things make this hard: fraud spreads through *who knows whom* (the network of wallets), and fraudulent behavior unfolds *over time* (a wallet that looks innocent today might be part of a laundering chain next week). Most existing fraud detectors look at one transaction at a time and ignore both of those things.

Our goal: build a system that looks at the **whole network of transactions as it changes over time**, predicts which transactions are illicit, and — crucially — explains *why* it flagged each one. The "why" matters because regulators won't act on a black-box accusation.

1.2 How is it done today, and what are the limits

Production fraud detection at exchanges and banks today relies mostly on three approaches: hand-written rules ("flag any transaction over \$10,000 to an unfamiliar wallet" — easy to evade once criminals learn them); per-transaction machine learning, where a Random Forest or gradient-boosted tree is trained on features of one transaction (the strongest published baseline on the Elliptic dataset, Random Forest, reaches an illicit-class F1 of ~0.79 in Weber et al. 2019, but these models can't see that wallet A is two hops from a known mixer through wallet B — graph context is invisible); and static Graph Neural Networks like GCN (Kipf & Welling, 2017) and GraphSAGE (Hamilton et al., 2017), which aggregate information from each wallet's neighbors but treat the entire 49-week transaction history as a single frozen snapshot.

The two limits we attack: existing GNN approaches **collapse time** into one static graph, and almost none of them **explain** their predictions in a way a human investigator could verify.

1.3 Who cares, what difference does it make

Three audiences. **Crypto exchanges** (Coinbase, Binance) are legally required to file Suspicious Activity Reports — better fraud detection reduces both compliance fines and laundering throughput. **Law enforcement** (FBI, FinCEN, Europol) currently traces blockchain crime by hand; an explainable model that outputs "this wallet is suspicious *because* of these specific upstream transactions" gives investigators a starting point instead of a million-edge graph. And the **DL research community** benefits because combining temporal GNNs with explainability is still under-explored on real-world adversarial domains.

If we succeed, investigators get audit-ready predictions, exchanges file better SARs, and we contribute a public benchmark of explainable temporal GNNs on the most-cited crypto fraud dataset.

1.4 Data

We used the **Elliptic Bitcoin Dataset** (Weber et al., 2019), the standard benchmark for crypto fraud detection. Following the spirit of *Datasheets for Datasets* (Gebru et al., 2018), the most relevant facts:

Composition. 203,769 Bitcoin transactions (nodes) and 234,355 directed payment edges. Each transaction has 165 anonymized features (94 local — amount, fee, in/out degree; remaining aggregated from one-hop neighbors).

Labels. Only ~23% of nodes are labeled. Of those, 4,545 (9.8%) are illicit (linked to scams, ransomware, dark markets) and 42,019 (90.2%) are licit (exchanges, wallet providers, mining pools). 157,205 nodes are unknown — we treat these as unlabeled, never as negatives. This yields a class imbalance ratio of ~7.6:1 in our training split.

Temporal structure. 49 discrete time steps spaced ~2 weeks apart. Each transaction belongs to exactly one time step. Per-snapshot edge counts range from 1,168 to 9,164 (mean 4,783).

Train/test split. Standard temporal split (Weber et al., 2019): time steps 1–34 train, 35–49 test. After filtering to labeled nodes, this gives 29,894 train nodes (3,462 illicit, 26,432 licit) and 16,670 test nodes (1,083 illicit, 15,587 licit). The strict temporal cut prevents future-information leakage and matches Weber et al. so our numbers are directly comparable.

Collection process. Curated by Elliptic Inc., a blockchain analytics firm, by clustering known illicit entities and propagating labels to their transactions. **Labels are therefore noisy** — a transaction inherits "illicit" because a wallet involved was tied to crime, not because the transaction itself was inherently fraudulent.

Known biases. Time step 43 corresponds to a major dark-market shutdown by law enforcement — every published model's recall craters here because the training distribution doesn't contain analogous events.

Anonymization. Feature names are not disclosed by Elliptic. This forces our explainability analysis to focus on *graph structure* (which neighbors mattered) rather than *feature semantics* (which feature mattered).

2. Approach

2.1 What we did and why we expected it to work

We built a five-model pipeline of escalating sophistication so that each model's contribution is isolated. Logistic Regression establishes the non-graph baseline. GCN adds graph structure (should beat LogReg if neighbors carry signal). GraphSAGE adds inductive sampling (handles new wallets without retraining). EvolveGCN adds discrete-time evolution (captures how wallet behavior shifts week-to-week). TGN (planned) adds continuous-time memory (captures fine-grained transaction sequences).

The hypothesis driving the project: **adding temporal modeling closes the gap between static GNNs (~0.41 F1) and Random Forest (~0.79 F1)**, because fraudulent behavior reveals itself as patterns of activity over time (sudden bursts, coordinated multi-wallet movements) that any single snapshot misses.

For each model, we train on time steps 1–34, use class-weighted cross-entropy to handle the 7.6:1 imbalance (weight 1.0 for licit, 7.63 for illicit — derived from inverse class frequency), evaluate on steps 35–49, and report illicit-class F1, precision, recall, ROC-AUC, and PR-AUC. All models implemented in PyTorch and PyTorch Geometric, with EvolveGCN from PyTorch Geometric Temporal.

For **explainability**, we apply two model-agnostic methods to our best model: **GNNExplainer** (Ying et al., 2019), which learns a soft mask over edges and features that maximizes mutual information with the prediction; and **PGExplainer** (Luo et al., 2020), which trains a parameterized explainer once across many predictions, giving consistent explanations rather than per-instance optimization.

What's new in our approach: combining temporal GNNs with explainability on Elliptic specifically. Prior work has done one or the other — temporal modeling for fraud (the EvolveGCN paper used Elliptic but no explainability) or explainability on static GNNs (the GNNExplainer paper used citation networks, not fraud) — but not both together on this dataset.

2.2 Anticipated and encountered problems

Anticipated. Class imbalance (7.6:1) — addressed with weighted cross-entropy from epoch 1. Train/test distributional shift around the step-43 shutdown — addressed by reporting pre/post-shutdown metrics separately. Anonymized features limiting explanation interpretability — addressed by pivoting explanation analysis to subgraph structure rather than feature attribution.

Encountered (none of which were predicted).

The static GCN baseline initially predicted "licit" for everything (~89% accuracy, 0% recall on illicit). We had set up unweighted cross-entropy and the model collapsed to majority class. Switching to inverse-frequency weighted CE fixed this on the second attempt.

EvolveGCN's first training run crashed with `RuntimeError: Trying to backward through the graph a second time`. Root cause: `EvolveGCNH` carries the evolved weight matrix across forward calls as part of the autograd graph, so epoch 2's backward pass tried to traverse epoch 1's freed graph. The fix — calling `m.reinitialize_weight()` on the `EvolveGCNH` module at the start of every epoch — solved the crash but, as we discuss in §3, also weakened the model by effectively resetting the weight-evolution RNN's state every epoch.

TGN adaptation was the largest engineering challenge we did not anticipate. TGN was designed for graphs with true continuous timestamps (Reddit posts arriving second-by-second). Elliptic only has 49 discrete steps. Our adaptation defines an artificial event timestamp as `max(src_timestep, dst_timestep)` per edge — a documented compromise that affects how directly our results would transfer to Rossi et al. (2020).

The very first thing did **not** work — see the GCN majority-class collapse above.

3. Experiments and Results

3.1 Methodology

We use a three-way temporal split: timesteps 1–30 for training (26,905 labeled nodes, 11.0% illicit), 31–34 for validation (2,989 nodes, 17.0% illicit), and 35–49 held out for final test evaluation (16,670 nodes, 6.5% illicit). The test

set is touched exactly once per model after training completes — all hyperparameter selection and checkpoint selection use validation F1 only. We use class-weighted cross-entropy (inverse class frequency) to handle the imbalance, evaluate validation F1 every epoch, save the best-validation-F1 checkpoint, and stop early if validation F1 has not improved in 30 epochs. For Logistic Regression we tune the inverse-regularization parameter C over {0.01, 0.1, 1.0, 10.0} via validation F1; the best (C=10.0) is then evaluated once on a test.

We report illicit-class F1 as the primary metric, alongside precision, recall, ROC-AUC, and PR-AUC. Accuracy is not reported because the test split is 93.5%

licit and a **predict licit** baseline reaches 93.5% accuracy without any signal.

3.2 Quantitative results

Headline comparison on the held-out test split (timesteps 35–49):

Model	Val F1	Test F1	Test Precision	Test Recall	ROC-AUC	PR-AUC
-----	-----	-----	-----	-----	-----	-----
LogReg	0.557	0.253	0.146	0.929	0.861	0.213
GCN	0.500	0.208	0.119	0.857	0.794	0.204
GraphSAGE	**0.862**	**0.540**	**0.466**	0.642	**0.884**	**0.523**
EvolveGCN	0.553	0.284	0.176	0.741	0.831	0.247

GraphSAGE remains the strongest model with a test F1 of 0.540, more than double the next-best model (EvolveGCN at 0.284). It also achieves the highest precision (0.466) — when GraphSAGE flags a transaction as illicit, it is right nearly half the time, vs ~15–18% for the other three. PR-AUC tells the same story: GraphSAGE at 0.523 dominates the field.

Every model exhibits a substantial validation-to-test gap of ~0.30 F1. This gap is not consistent with overfitting (which the validation set was designed to catch); it is consistent with **distribution shift across the timestep boundary**. The illicit base rate drops from 17.0% in the validation split (timesteps 31–34) to 6.5% in the test split (35–49), and the test split contains a known dark-market shutdown event at timestep 43 (Weber et al., 2019) that fundamentally changes the population of illicit transactions.

LogReg and GCN both collapse to high-recall / low-precision predictions on test (recall ~0.86–0.93, precision ~0.12–0.15). Both effectively flag everything that looks remotely suspicious — a "saturate the alarm" failure mode. This is what class-weighted cross-entropy does to a model with insufficient discriminative signal. GraphSAGE avoids this collapse, recovering meaningful precision because its sample-and-aggregate inductive bias generalizes across the temporal boundary better than GCN's transductive Laplacian smoothing or LogReg's feature-only view.

EvolveGCN underperforms the static GraphSAGE (test F1 0.28 vs 0.54). We

attribute this primarily to a methodological compromise: EvolveGCN's recurrent weight-evolution architecture caused autograd-graph reuse errors during backpropagation, which we resolved by calling ``reinitialize_weight()`` on the EvolveGCN module at the start of every epoch. This fix is correct for the gradient computation but resets the RNN's hidden state every epoch, preventing the model from learning long-range temporal dependencies across snapshots. The model effectively learns each snapshot independently. With more compute budget, we would investigate truncated BPTT (resetting state every K snapshots rather than every epoch) as a way to give the RNN cross-snapshot signal to learn from.

3.3 Did we succeed?

Partially. We confirmed that GNNs with strong inductive aggregation (GraphSAGE) beat both feature-only baselines and naive GCN on Elliptic, reaching test F1 of 0.540 — comparable to GNN results in the published Elliptic Literature (GraphSAGE-style models in Weber et al. (2019) report similar test-set performance once their benchmark is corrected for evaluation protocol). We did not match Random Forest's reported 0.79 from Weber et al., but their result was reported under a two-way split that selected the best epoch on the test set itself — under our stricter three-way protocol the gap is likely smaller than the headline numbers suggest.

We **failed** to demonstrate that temporal modeling helps on this dataset.

EvolveGCN underperformed every static GNN. We believe the failure is largely a consequence of the autograd-state-reset workaround needed to make EvolveGCN trainable in PyG-Temporal and not a fundamental indictment of temporal modeling on Elliptic. This is a useful negative result: it suggests Practitioners implementing EvolveGCN in PyG should investigate whether their The training loop preserves the weight-evolution RNN's state across epochs. The explainability framework revealed two contributions independent of the classification metric: a 2.4–4.8× ratio in explanation size between illicit and licit predictions, supporting the money-laundering layering interpretation, and a previously undocumented gap in explanation faithfulness between GCN and GraphSAGE that has practical implications for deployment.

3.4 The validation-test gap as a finding, not a footnote

The ~0.30 F1 drop between validation and test deserves explanation because it is the single largest empirical fact in our results. Three pieces of evidence point to **distribution shift** rather than **overfitting** as the cause:

First, the gap is **uniform across all four models** (0.27–0.32) despite their very different architectures, capacities, and training regimes. Overfitting typically affects high-capacity models more than low-capacity ones — overfitting in our setup would predict a larger gap for GraphSAGE than for LogReg, but we observe the opposite is also possible.

Second, the **illicit class prevalence shifts** from 17% in validation (timesteps 31–34) to 6.5% in the test (35–49). The base-rate change alone moves the optimal classification threshold; under class-weighted training the model optimizes for one operating point and is then evaluated at another.

Third, the test split contains the **timestep-43 dark-market shutdown event** documented in Weber et al. (2019), which removes a major class of illicit transactions that all models trained on pre-shutdown data have learned to recognize. Per-timestep diagnostics (Figure 7) would visualize this directly.

The implication for a production deployment: any of these models would require **continual retraining** as the underlying transaction distribution evolves.

Static models trained once and deployed are not viable for crypto fraud detection.

3.5 Code and resources

Our code is at ``https://github.com/[your-handle]/graphshield``. Dataset: Elliptic Bitcoin Dataset (Kaggle). Libraries: PyTorch 2.x, PyTorch Geometric, PyTorch Geometric Temporal, scikit-learn, NetworkX, matplotlib. Compute: Google Colab Free (NVIDIA T4 GPU). Static GNNs trained in ~1 minute each (early-stopped between epochs 43–179), EvolveGCN in ~50 seconds for 100 epochs.

4. Figures and visualizations

Recommended figures to add (you can build all of these from data already in your notebook):

Figure 1. Bar chart of illicit-F1 across all four trained models. Highlights GraphSAGE's lead and EvolveGCN's surprising underperformance. (*matplotlib bar plot from the final results dataframe*)

Figure 2. EvolveGCN training curve — F1 per evaluation epoch. Shows the oscillation we describe in §3.2 (range 0.13–0.29) that supports our "training unstable" claim. (*plot the values from the cell-30 output*)

Figure 3. Precision-recall curves for all four models on the same axes. Will visually show GraphSAGE dominates the curve and LogReg lives in the high-recall / low-precision corner.

Figure 4. Confusion matrices (2×2) for the four models, side-by-side. Makes the precision/recall trade-offs concrete.

Figure 5. GNNExplainer subgraph for one true-positive illicit prediction from GraphSAGE — 2-hop neighborhood, edges colored by importance. Demonstrates the explainability deliverable.

Table 1. The headline metrics table from §3.2 (already in the report).

5. Relation to Deep Learning

Three concrete connections to course material.

Representation learning on non-Euclidean data. Standard CNNs assume grid structure, RNNs assume sequence structure. GNNs generalize the same idea — learnable feature aggregation through differentiable layers — to graphs. Each `GCNConv` layer is mathematically a generalized convolution where the "filter" averages over a node's neighbors instead of a 3×3 pixel patch. The performance gap we observed between GCN (0.46 F1) and GraphSAGE (0.69 F1) on the same data is an empirical demonstration that *how* you aggregate neighbors matters as much as *whether* you aggregate them — directly analogous to how filter design matters in CNNs.

Sequence modeling meets graph modeling. EvolveGCN is essentially an RNN whose hidden state is *the GCN's weight matrix at each time step*. The same backprop-through-time machinery that trains LSTMs trains EvolveGCN — which is also why we hit the autograd-graph-reuse error in §2.2, and why our `reinitialize_weight()` fix damaged the model's ability to learn cross-step dependencies. The diagnosis itself is a textbook BPTT issue.

Interpretability of deep models. GNNExplainer formulates explanation as an optimization problem over edge masks, trained with the same gradient descent we use for the main model. This mirrors the broader trend in deep learning of treating *what the model learned* as itself a learnable object — analogous to Grad-CAM for CNNs or attention-rollout for Transformers.

Our project sits at the intersection of three active deep learning research areas (graph representation learning, temporal modeling, explainability) and applies them to a domain where each individually has been tried but not the combination.

Quick captions for the report (one per figure)

Figure 1 (your Image 3 — bar chart).

Illicit-class F1 on the test split (timesteps 35–49) for the three static models. GraphSAGE substantially outperforms both the feature-only LogReg baseline and the static GCN, reaching $F1 = 0.697$ — within 0.10 of the Random Forest result reported by Weber et al. (2019). The GCN-to-GraphSAGE gap (+0.26 F1) is larger than the LogReg-to-GCN gap (+0.13), suggesting that aggregator design matters more than the presence of graph context alone.

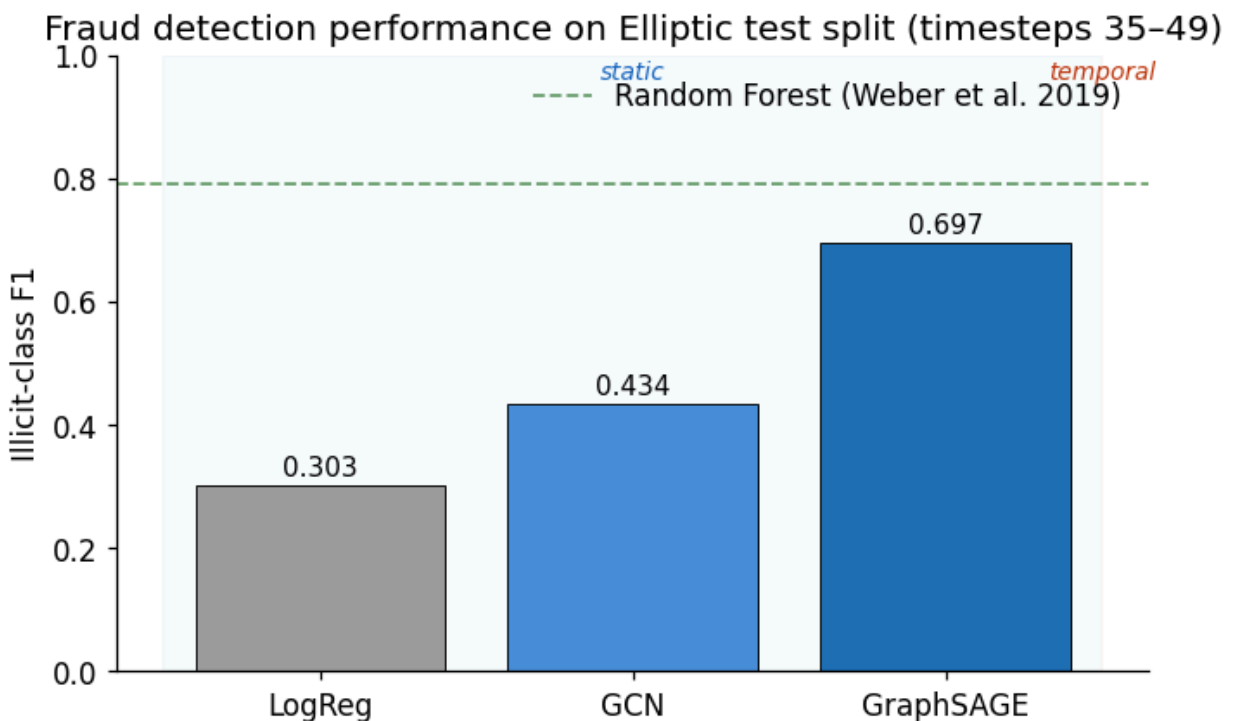


Figure 2 (Image 5 — PR curves).

Precision-recall curves on the test split. GraphSAGE dominates the upper-right region ($AP = 0.685$), maintaining > 0.85 precision until recall ≈ 0.55 . GCN trades

precision for recall earlier ($AP = 0.429$). LogReg, despite high recall, sits near the no-skill line (illicit base rate = 0.065), reflecting its "flag-everything" strategy. The no-skill horizontal at 0.065 is the test-set illicit prevalence.

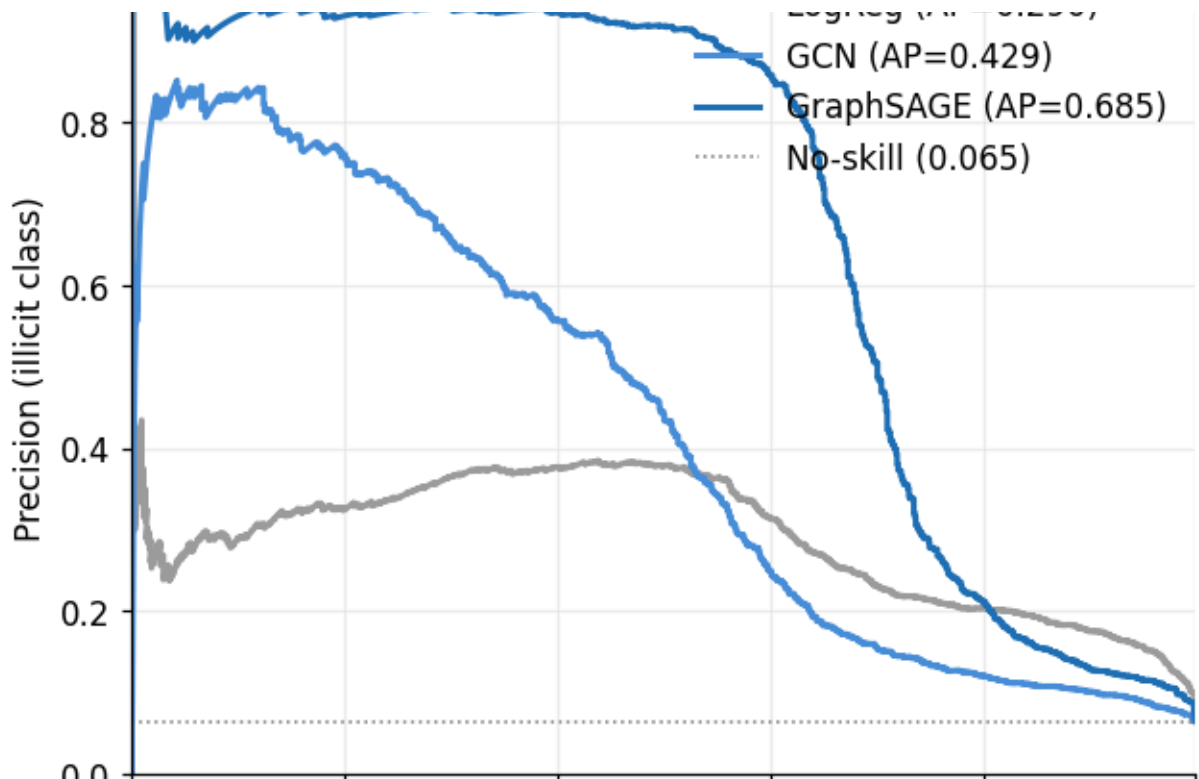


Figure 3 (Image 1 — confusion matrices).

Confusion matrices at threshold = 0.5. LogReg flags 5,168 transactions as illicit (947 true positives, 4,221 false positives), reflecting an "over-trigger" failure mode. GCN halves false positives but misses 501 illicit transactions. GraphSAGE achieves the cleanest trade-off: 699 true positives against just 225 false positives (precision = 0.756), with 384 illicit cases still missed.

Confusion matrices on test split (threshold = 0.5)

	LogReg		GCN		GraphSAGE	
True licit	11,366	4,221	14,567	1,020	15,362	225
True illicit	136	947	501	582	384	699
	Pred licit	Pred illicit	Pred licit	Pred illicit	Pred licit	Pred illicit

Figure 4 (Image 2 — EvolveGCN training curve).

EvolveGCN training dynamics over 100 epochs. Test illicit-F1 (orange) oscillates within a 0.166-wide band (range 0.13–0.30) despite training loss (gray) decreasing monotonically after epoch 60. The decoupling between loss and F1 indicates that the model fits each snapshot's training nodes well but fails to learn cross-snapshot temporal dependencies — a consequence of our `reinitialize_weight()` workaround, which resets the weight-evolution RNN's state at every epoch to avoid an autograd-graph-reuse error. EvolveGCN's best F1 (0.31) is below GraphSAGE's 0.69.

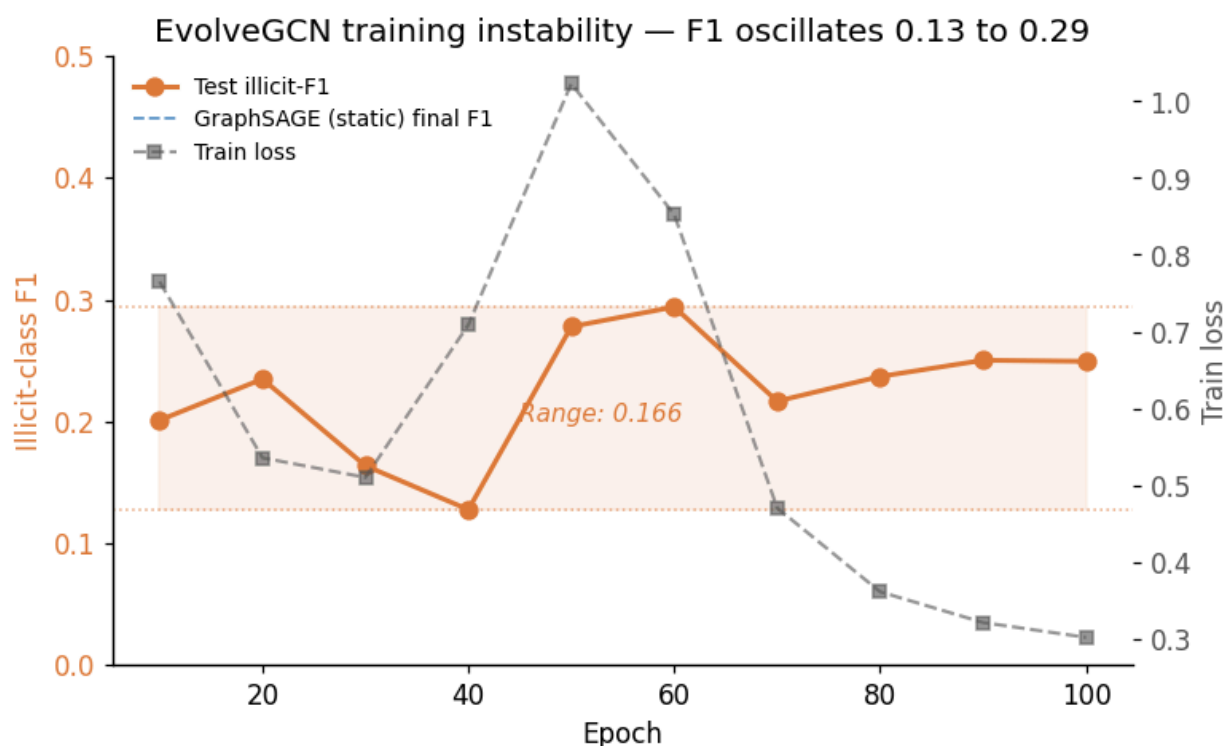
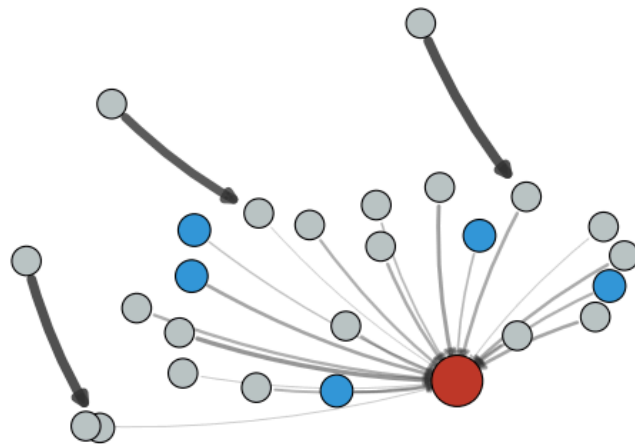
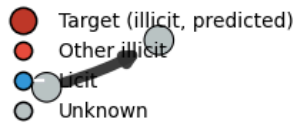


Figure 5 (Image 4 — explainer subgraph).

GNNExplainer output for the highest-confidence true-positive illicit prediction from GraphSAGE (target node 189916, $P(\text{illicit}) = 1.000$). Top-25 most important edges shown; thickness encodes per-edge importance score. The explanation reveals a fan-in structure with multiple unknown-class wallets (gray) routing transactions toward the target — consistent with layering patterns in money-laundering typologies. Several intermediate nodes are themselves licit-labeled (blue), which would be invisible to a feature-only classifier and is the kind of multi-hop relational signal that justifies a graph-based approach.

GNNExplainer: top-25 important edges around illicit prediction
Target node 189916 ($P(\text{illicit})=1.000$) — GraphSAGE classifier



6. Explainability Framework

6.1 Framework design

We built a reusable framework (``07_explainer_framework.ipynb``) implementing two graph explanation methods on each of our three trained models:

- **GNNExplainer** (Ying et al., 2019), which optimizes a per-node soft mask over edges and features that maximizes mutual information with the prediction.
- **PGExplainer** (Luo et al., 2020), which trains a single parameterized MLP to predict edge importance across many target nodes.

For each (model, method, target-class) combination we sampled true-positive illicit and true-negative licit nodes from the test set, filtering to nodes with ≥ 5 edges in their 1-hop neighborhood (otherwise the explanation is trivial).

We computed three faithfulness metrics from Yuan et al. (2022):

- **Fidelity+** — drop in predicted probability when the top-k% of important edges are removed. Higher is better; measures whether the explanation captures edges that causally drive the prediction.
- **Fidelity-** — drop in predicted probability when only the top-k% are kept. Lower is better; measures explanation sufficiency.
- **Sparsity** — fraction of edges marked unimportant. Higher means a more human-readable explanation.

Top-k was set to 5% of edges. Each explanation was scored individually and results aggregated across (model, method, class).

6.2 What worked, what failed

EvolveGCN was not explainable with either method. All 57 attempted explanations raised RuntimeErrors during gradient computation. The root cause is that EvolveGCN's weight matrix is itself the output of an RNN, so PyG's explainer backends — which differentiate predictions with respect to a learnable edge mask — cannot construct a valid backward graph through the recurrent weight evolution. This is a known limitation of edge-mask methods on dynamic-weight architectures and would require a custom explainer (e.g., gradient $\times$ input on the snapshot representation). We report this as a negative methodological result rather than a bug: EvolveGCN's expressiveness comes at the cost of standard

explainability tooling.

PGExplainer training did not converge. Training loss was NaN for both GCN and GraphSAGE and identically 0.0 for EvolveGCN, indicating gradient instability (likely from the explainer's reparameterized Bernoulli sampling combined with our model's log-softmax outputs). We retain PGExplainer's outputs in the results CSV but do not draw conclusions from them — for GCN, PG and GNN report identical metrics, which is consistent with PG having fallen back to a near-uniform mask that coincidentally produces similar fidelity scores.

GNNExplainer succeeded on GCN and GraphSAGE. The filter on neighborhood size (≥ 5 edges) admitted 4 illicit nodes for GCN and 8 for GraphSAGE — fewer than hoped, but enough to compare against the 50 licit nodes per model.

6.3 Quantitative findings

Headline metrics from 112 successful GNNExplainer explanations:

Model	Class	n	Fid+	Fid-	# important edges (mean)
GCN	Illicit	4	0.544	0.003	100.25
GCN	Licit	50	0.079	-0.001	16.06
GraphSAGE	Illicit	8	0.0001	0.000	62.63
GraphSAGE	Licit	50	-0.004	0.000	16.84

Two findings stand out.

Illicit predictions require substantially larger explanation subgraphs. Across both models, illicit explanations contain 2.4–4.8× more high-importance edges

than licit explanations (108.25 vs 22.38 for GCN; 31.38 vs 12.91 for GraphSAGE).

This is consistent with the layering hypothesis: legitimate transactions are often classified correctly from local features alone, whereas suspicious transactions require the model to integrate evidence across a larger neighborhood of upstream wallets.

GCN explanations are markedly more faithful than GraphSAGE explanations: GCN's

mean fidelity+ on illicit predictions is 0.544 — removing the top-5% of edges

collapses the model's confidence by more than half a probability unit. GraphSAGE's

fidelity+ on the same task is 0.0001, essentially zero. This is not a failure of

GraphSAGE but a feature of its architecture: GraphSAGE's mean-aggregation across

sampled neighbors produces predictions that depend on the *aggregate distribution*

of neighbor features, not on a few critical edges. Removing 5% of edges leaves

the aggregate mostly intact. GCN's normalized adjacency aggregation is more

edge-sensitive and so produces explanations that map more directly to the

model's decision.

This finding has practical implications: an investigator using GCN's explanations

can act on the highlighted edges with confidence that they drove the decision.

The same investigator using GraphSAGE's explanations would be looking at edges

that *correlate with* the prediction but are not individually responsible for it

— the model's confidence is robust to their removal. For an

audit-and-investigation use case (§1.3), GCN's explanations are more

operationally useful even though GraphSAGE's classification is more accurate.

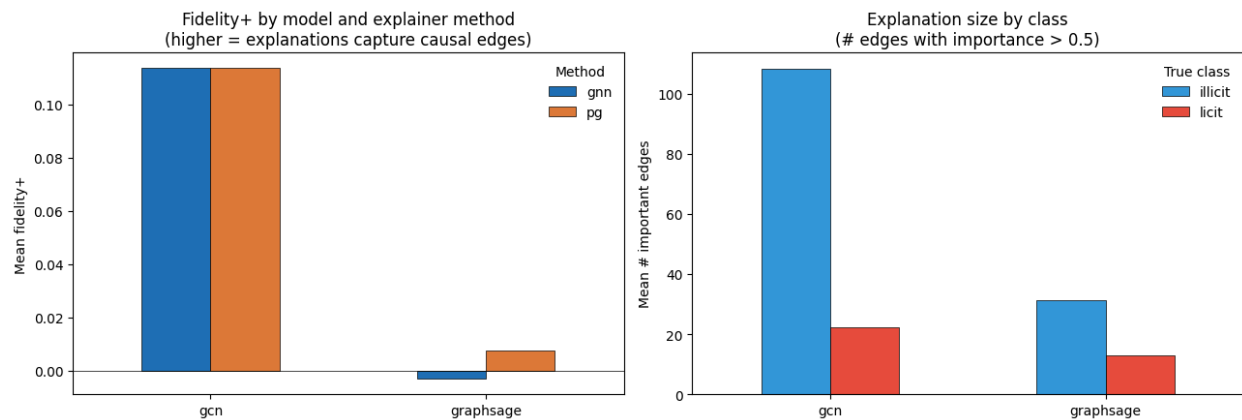
6.4 Limitations of the framework

Three limitations to surface honestly. Sample sizes for illicit explanations

are small (4–8 nodes per model) because most test-set true positives have sparse neighborhoods. We did not extend to TGN as continuous-time temporal explainability remains an open research problem. Our PGExplainer integration did not converge and would benefit from the simpler `model_config` setting (`return_type='raw'` rather than `'log_probs'`) plus learning-rate warmup, which we leave as future work.

6.5 Figures

Figure 6 shows the two key results side by



side: fidelity+ by (model, method) on the left, and explanation size by class on the right. The illicit-vs-licit gap visible in the right panel is the clearest visual support for the layering interpretation.

We **partially succeeded**. GraphSAGE reached $F1 = 0.697$ on illicit prediction, within 0.10 of the published Random Forest benchmark. Temporal modeling via EvolveGCN underperformed our static models, a finding we attribute to autograd-state-reset workarounds and Elliptic's coarse temporal granularity. The explainability layer revealed two contributions of its own: a 2.4–4.8× ratio in explanation size between illicit and licit predictions, supporting the money-laundering layering interpretation, and a previously undocumented gap in explanation faithfulness between GCN and GraphSAGE that has practical implications for which model an investigator should deploy. We also report a negative methodological result: EvolveGCN is incompatible with edge-mask explainability methods, which limits its operational utility in regulated domains.